

6-2 Project One

Malgorzata Debska

Southern New Hampshire University

CS-300 Analysis and Design

David Ostrowski

VECTOR PSEUDOCODE

```
// Define Course object
class Course:
    attributes:
        title
        prerequisites

// Initialize an empty vector to store Course objects
courses_vector = new Vector()

// Open the file and read data
function read_file(file_name):
    open file_name
    for each line in file:
        course_info = parse_line(line)
        if course_info is not null:
            course = create_course(course_info)
            courses_vector.append(course)
        else:
            handle_formatting_error(line)

// Parse each line and check for formatting errors
function parse_line(line):
```

```
split line by delimiter (e.g., comma)
```

```
if number of elements != expected:
```

```
    return null
```

```
else:
```

```
    return course_info
```

```
// Create Course object
```

```
function create_course(course_info):
```

```
    title = course_info[0]
```

```
    prerequisites = course_info[1:]
```

```
    return new Course(title, prerequisites)
```

```
// Print course information and prerequisites
```

```
function print_course_info(course_title):
```

```
    for each course in courses_vector:
```

```
        if course.title == course_title:
```

```
            print "Title:", course.title
```

```
            print "Prerequisites:", course.prerequisites
```

```
            return
```

```
    print "Course not found"
```

HASH TABLE PSEUDOCODE

```
// Define Course object
```

```
class Course:
```

```
    attributes:
```

```
        title
```

prerequisites

```
// Initialize an empty hash table to store Course objects
```

```
courses_hash_table = new HashTable()
```

```
// Open the file and read data
```

```
function read_file(file_name):
```

```
    open file_name
```

```
    for each line in file:
```

```
        course_info = parse_line(line)
```

```
        if course_info is not null:
```

```
            course = create_course(course_info)
```

```
            courses_hash_table.insert(course.title, course)
```

```
        else:
```

```
            handle_formatting_error(line)
```

```
// Parse each line and check for formatting errors
```

```
function parse_line(line):
```

```
    split line by delimiter (e.g., comma)
```

```
    if number of elements != expected:
```

```
        return null
```

```
    else:
```

```
        return course_info
```

```
// Create Course object
```

```
function create_course(course_info):
```

```
    title = course_info[0]
```

```
prerequisites = course_info[1:]  
return new Course(title, prerequisites)
```

```
// Print course information and prerequisites  
function print_course_info(course_title):  
    course = courses_hash_table.lookup(course_title)  
    if course is not null:  
        print "Title:", course.title  
        print "Prerequisites:", course.prerequisites  
    else:  
        print "Course not found"
```

TREE PSEUDOCODE

```
// Define Course object
```

```
class Course:  
    attributes:  
        title  
        prerequisites
```

```
// Define Node object
```

```
class Node:  
    attributes:  
        data (Course object)  
        left_child  
        right_child
```

```
// Initialize an empty tree to store Course objects
```

```
courses_tree = new BinaryTree()
```

```
// Open the file and read data
```

```
function read_file(file_name):
```

```
    open file_name
```

```
    for each line in file:
```

```
        course_info = parse_line(line)
```

```
        if course_info is not null:
```

```
            course = create_course(course_info)
```

```
            courses_tree.insert(course)
```

```
        else:
```

```
            handle_formatting_error(line)
```

```
// Parse each line and check for formatting errors
```

```
function parse_line(line):
```

```
    split line by delimiter (e.g., comma)
```

```
    if number of elements != expected:
```

```
        return null
```

```
    else:
```

```
        return course_info
```

```
// Create Course object
```

```
function create_course(course_info):
```

```
    title = course_info[0]
```

```
    prerequisites = course_info[1:]
```

```
return new Course(title, prerequisites)
```

```
// Print course information and prerequisites
```

```
function print_course_info(course_title, node):
```

```
    if node is not null:
```

```
        if node.data.title == course_title:
```

```
            print "Title:", node.data.title
```

```
            print "Prerequisites:", node.data.prerequisites
```

```
            return
```

```
        else if course_title < node.data.title:
```

```
            print_course_info(course_title, node.left_child)
```

```
        else:
```

```
            print_course_info(course_title, node.right_child)
```

```
    print "Course not found"
```

FILE HANDLING AND COURSE OBJECT CREATION

```
// Open the file and read data
```

```
function read_file(file_name):
```

```
    open file_name
```

```
    for each line in file:
```

```
        course_info = parse_line(line)
```

```
        if course_info is not null:
```

```
            // For Vector and Hash Table
```

```
            course = create_course(course_info)
```

```
            // For Tree
```

```
            course = create_course_object(course_info)
```

```

        // Perform operations based on data structure

// Parse each line and check for formatting errors
function parse_line(line):
    split line by delimiter (e.g., comma)
    if number of elements != expected:
        return null
    else:
        return course_info

// Create Course object
function create_course(course_info):
    title = course_info[0]
    prerequisites = course_info[1:]
    return new Course(title, prerequisites)

// Create Course object for Tree
function create_course_object(course_info):
    title = course_info[0]
    prerequisites = course_info[1:]
    return new Node(new Course(title, prerequisites))

```

MENU PSEUDOCODE

Menu:

```

while True:
    PrintMenuOptions()
    choice = GetUserChoice()

```



```
if choice == "1":
    LoadDataStructure()
elif choice == "2":
    PrintCourseList()
elif choice == "3":
    PrintCourse()
elif choice == "4":
    ExitProgram()
else:
    print("Invalid choice. Please select again.")
```

PrintMenuOptions():

```
print("1. Load Data Structure")
print("2. Print Course List")
print("3. Print Course")
print("4. Exit")
```

GetUserChoice():

```
return input("Enter your choice: ")
```

ExitProgram():

```
print("Exiting program.")
exit()
```

LoadDataStructure():

```
fileName = input("Enter file name: ")
file = OpenFile(fileName)
```

```

if file is not null:
    data = ReadDataFromFile(file)
    // Load data into chosen data structure

```

PSUDOCODE FOR PRINTING ALPHANUMERICALLY SORTED COURSE LIST

PrintAlphanumericCourseList():

```

data_structure = ChooseDataStructure()
if data_structure is not null:
    sorted_courses = data_structure.SortCoursesAlphabetically()
    data_structure.PrintCourseList()

```

Vector

Operation	Cost per Line	Times Executed (for n courses)	Total Cost
Open the file	1	1	1
Read a line from the file	1	n	n
Parse the line	1	n	n
Create a course object	1	n	n
Append the course object to the vector	1	n	n
Print a course title	1	n	n
Print course information and prerequisites	1	n	n
Total			6n + 1

Hash Table

Operation	Cost per Line	Times Executed (for n courses)	Total Cost
Open the file	1	1	1
Read a line from the file	1	n	n
Parse the line	1	n	n
Create a course object	1	n	n
Insert the course object into the hash table	O(1)	n	n
Print a course title	1	n	n
Print course information and prerequisites	1	n	n
Total			5n + 1

Tree

Operation	Cost per Line	Times Executed (for n courses)	Total Cost
Open the file	1	1	1
Read a line from the file	1	n	n
Parse the line	1	n	n
Create a course object	1	n	n
Insert the course object into the tree	O(log n)	n	n log n
Print a course title	1	n	n
Print course information and prerequisites	1	n	n
Total			3n + n log n + 1

Analysis of Data Structures:

- **Vector:**
 - Advantages: Simple implementation maintains insertion order, suitable for small to medium-sized datasets.
 - Disadvantages: Slower search time ($O(n)$ for finding courses), especially as the dataset grows.
- **Hash Table:**

- Advantages: Fast lookup time ($O(1)$ on average), suitable for large datasets with frequent lookups.
- Disadvantages: Does not maintain insertion order, may have collisions which can degrade performance (though this can be mitigated with good hashing functions).
 - **Tree:**
- Advantages: Provides sorted order traversal, suitable for scenarios where courses need to be accessed in a sorted manner, such as printing courses in alphanumeric order.
- Disadvantages: Slower insertion time compared to hash table ($O(\log n)$), may become unbalanced leading to degraded performance.

Recommendation:

Based on the analysis, the recommended data structure for this scenario would be the **Hash Table**. It offers fast lookup time ($O(1)$ on average) which is crucial for quickly retrieving course information and prerequisites. Additionally, the hash table can handle large datasets efficiently and is well-suited for scenarios where the order of insertion does not matter. While the tree provides sorted order traversal, the performance advantage of the hash table in terms of lookup time outweighs this benefit, especially considering the frequent lookups required in this scenario.